

AIQT CLI

Milestone 4 Build Specification v0.4

Agent Handoff Packet Engine

Build-planning document for implementing AIQT Milestone 4. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 4 Build Specification
Version	v0.4
Milestone	M4 - Agent Handoff Packet Engine
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Depends On	M1 - Canonical State Foundation, tag m1-canonical-state-foundation M2 - Project Context Capture, tag m2-project-context-capture M3 - Work Graph Planning Foundation, merge commit f922a47, tag m3-work-graph-planning
Primary Agent Target	Claude Code or equivalent coding agent
Implementation Status	Ready for final review before build handoff

Document status: Ready for final review before build handoff. v0.4 aligns aiqt next fallback behavior with M1-M3 navigation and aligns the packet template with the MVP base structure plus the M4 Dependencies extension.

Contents

- 1. Source Alignment
- 2. Milestone Objective
- 3. Agent Packet Model Decision
- 4. Commands in Scope
- 5. Out of Scope
- 6. Behavioral Position in the Workflow
- 7. Handoff Preconditions
- 8. Work Unit Selection Rules
- 9. Agent Handoff Gate
- 10. Packet Context Selection
- 11. Packet Format
- 12. Canonical Mutation Rules
- 13. State Transition Rules
- 14. Runlog Rules
- 15. Command Behavior
- 16. Data and Schema Contracts
- 17. Error Handling and Exit Codes
- 18. Repository Implementation Scope
- 19. Runtime and Package Impact
- 20. Testing Scope
- 21. Implementation Tasks
- 22. Definition of Done
- 23. Agent Execution Packet
- Appendix A - Minimum Test Matrix
- Appendix B - Example Agent Packet

1. Source Alignment

This document is derived from AIQT MVP Specification v0.6, AIQT Technical Architecture Specification v0.3, and the milestone sequence already established by M1, M2, and M3. It assumes M1, M2, and M3 are complete on main. M3 is merged at commit f922a47 and tagged as m3-work-graph-planning. The repository is expected to provide a populated workGraph in state.json with at least one ready work unit after aiqt plan.

The governing design remains:

```
state -> graph -> packet -> checkpoint  
not:  
spec -> milestone file -> task file -> review file
```

M4 implements the packet step. It must not perform checkpoint capture, review, export, agent execution, or source-code modification.

2. Milestone Objective

Milestone 4 implements the full aiqt next agent handoff behavior. It turns a planned work graph into one bounded, paste-ready execution packet for a coding agent, and updates workflow state so AIQT knows which work unit is now in progress.

Milestone 4 must prove that AIQT can:

- read project.json and state.json from an initialized and planned AIQT project;
- select one ready work unit from state.workGraph deterministically;
- validate that the selected work unit passes the agent handoff gate;
- generate a compact agent execution packet for exactly one work unit;
- include only relevant project context, constraints, acceptance criteria, dependencies, suggested files, validation commands, and required output instructions;
- mark the selected work unit as in_progress;
- set currentWorkUnitId and currentMilestoneId;
- set projectStatus and milestone status to in_progress where applicable;
- store lastAgentPacket metadata in state.json;
- append deterministic runlog events;
- return stable human-readable and JSON output;
- preserve all M1, M2, and M3 behavior.

M4 is not an agent runner. It produces the handoff packet that the human can paste into Claude Code or another coding agent.

3. Agent Packet Model Decision

M4 generates a markdown-like agent execution packet as command output. The packet is generated from canonical state, but it is not stored as a persistent markdown file by default.

Decision: aiqt next should mutate workflow state when it selects a ready work unit. The command is not just a preview. It begins an execution cycle by marking the selected work unit in_progress and recommending aiqt checkpoint as the next command.

Reason: M3 leaves currentWorkUnitId null. M4 is the point where AIQT moves from planned work to an active agent execution cycle.

4. Commands in Scope

aiqt next
aiqt next --json

The default human-readable output of aiqt next should be the agent packet itself when the command succeeds. JSON mode wraps the same packet in the existing CommandResult contract.

aiqt next --copy is out of scope for M4. Clipboard behavior is useful but creates cross-platform testing friction and is not necessary for the first packet engine milestone.

5. Out of Scope

- aiqt checkpoint implementation;
- checkpoint capture;
- review engine implementation;
- export engine implementation;
- aiqt next --copy;
- agent execution or process spawning;
- Claude Code, OpenAI, Codex, or GitHub Copilot API integration;
- GitHub integration;
- source-code modification of user projects;
- markdown file generation by default;
- AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, or .tasks/;
- replanning or graph mutation beyond status/current-work updates;
- accepting checkpoint results or validation output from an agent;
- multi-work-unit packet generation.

6. Behavioral Position in the Workflow

M1: init/status/next -> M2: update -> M3: plan -> M4: next agent packet -> M5: checkpoint -> M6: review/export

After M3, state.workGraph contains milestones, workUnits, and dependencies. state.projectStatus is planned, currentMilestoneId points to the milestone of the first ready work unit, currentWorkUnitId is null, and nextRecommendedCommand is aiqt next.

After M4, one work unit is in_progress. The user should send the packet to Claude Code, let the coding agent perform the work outside AIQT, and then run aiqt checkpoint in M5.

7. Handoff Preconditions

Before generating an agent packet, aiqt next must validate these workflow preconditions:

Precondition	Failure behavior
.aiqt/ exists and canonical files are readable	Exit code 3, status failed, recommend aiqt init or repair state.
project.json and state.json validate against schema and version rules	Exit code 3, status failed.
state.workGraph is empty and planningContextReady is false	Exit code 2, status blocked, recommend aiqt update. No mutation.
state.workGraph is empty and planningContextReady is true	Exit code 2, status blocked, recommend aiqt plan. No mutation.
No current work unit is already in progress	Exit code 2, status blocked, recommend aiqt checkpoint.

At least one ready work unit exists	Exit code 2, status blocked, recommend aiqt review.
Selected work unit fails content/quality handoff gate	Exit code 1, status failed, no mutation, recommend aiqt review.
Selected work unit has invalid milestone or dependency references	Exit code 3, status failed, no mutation, recommend aiqt review.

Exit code 1 means the selected work unit exists but is not acceptable for handoff quality. Exit code 3 means the canonical work graph is structurally invalid or contains broken references.

8. Work Unit Selection Rules

The selected work unit is the first ready work unit by `state.workGraph.workUnits` order. M4 must not reorder work units or infer priority from milestone title, ID, dependencies, or suggested files.

Selection algorithm:

1. Read `state.workGraph.workUnits` in stored order.
2. Find the first work unit with status = ready.
3. Verify the work unit has a valid milestoneId that exists in `state.workGraph.milestones`.
4. Verify the work unit passes the agent handoff gate.
5. Select that single work unit for packet generation.
6. Do not include other ready work units as future tasks unless directly referenced by dependencies.

If `state.currentWorkUnitId` is non-null before selection, M4 must not select another work unit. The workflow is already mid-execution and must proceed through aiqt checkpoint.

9. Agent Handoff Gate

The agent handoff gate prevents AIQT from producing vague or unsafe packets. It is an internal validator used by aiqt next.

Gate rule	Failure
Selected work unit status must be ready	ValidationFailed, exit code 1.
Work unit title and objective must be non-empty	ValidationFailed, exit code 1.
scope must contain at least one non-empty item	ValidationFailed, exit code 1.
outOfScope must contain at least one non-empty item	ValidationFailed, exit code 1.
acceptanceCriteria must contain at least one non-empty item	ValidationFailed, exit code 1.
validationCommands must contain at least one non-empty item	ValidationFailed, exit code 1.
milestoneId must reference an existing milestone	Invalid state, exit code 3.
dependency IDs referenced by the selected work unit must exist	Invalid state, exit code 3.
agentContextRefs should resolve to known project records when possible	Unresolved refs always produce warnings in M4; they do not block packet generation.

The packet must target exactly one work unit. A packet containing multiple work units, a full milestone plan, or the full project history fails the handoff gate by design.

Unresolved `agentContextRefs` rule: unresolved refs always produce warnings in M4 with severity = medium, area = context, and `agentCanFix` = false. M4 has no `requiredContextRefs` field, so there is no hard-blocking context-reference mode yet. Future milestones may add required context references explicitly.

10. Packet Context Selection

M4 must keep the packet compact. It should include enough context for the agent to execute the selected work unit, but it must not dump all project history.

Always include:

- project.project.objective;
- project.project.targetUsers if present;
- selected milestone ID, title, and objective;
- selected work unit ID, title, objective, scope, outOfScope, acceptanceCriteria, suggestedFiles, validationCommands;
- project.context.constraints;
- project.context.technologyPreferences;
- project.context.businessRules;
- dependency summary for dependencies directly attached to the selected work unit;
- required agent output contract.

Include only when referenced by selectedWorkUnit.agentContextRefs:

- requirements;
- decisions;
- risks;
- assumptions;
- openQuestions;
- architecture notes beyond the global architectureNotes list if future schemas add them.

Do not include:

- unrelated milestones;
- unrelated future work units;
- full runlog history;
- full checkpoint history;
- human planning commentary;
- markdown export wrappers;
- previous chat transcripts.

11. Packet Format

The human-readable success output of aiqt next should be the agent packet text. Avoid decorative terminal output around the packet so it can be copied directly into Claude Code. M4 uses the MVP packet template as the base format and intentionally extends it with a dedicated Dependencies section because dependencies become available after M3. M4 preserves the standalone Constraints section and keeps Acceptance Criteria before Suggested Files / Areas.

```
# AGENT EXECUTION PACKET
```

```
## Role
```

You are implementing one bounded work unit for this project.

Project Objective

<project.objective>

Current Work Unit

ID: <workUnitId>

Milestone: <milestoneId> - <milestoneTitle>

Title: <workUnitTitle>

Objective: <workUnitObjective>

Scope

- <scope item>

Out of Scope

- <out-of-scope item>

Relevant Context

- Target users: <target users>

- Technology preference: <technology preference>

- Business rule: <business rule>

- Requirement [REQ-001]: <title> - <description>

Acceptance criteria: <criterion 1>; <criterion 2>

- Decision [D001]: <decision>

Reason: <reason>

Impact: <impact>

- Risk [RISK-001]: <title> - Severity: <severity>

Mitigation: <mitigation or "None recorded">

- Assumption [ASM-001]: <statement>

Reason: <reason or "None recorded">

- Open Question [Q001]: <question>

Impact: <impact>

Status: <status>

Answer: <answer or "Unanswered">

Constraints

- <constraint>

Dependencies

- <dependency summary or "None">

Acceptance Criteria

- <criterion>

Suggested Files / Areas

- <file or area>

Validation Commands

```bash

<command>

```

Required Agent Output

Return:

1. Summary of implementation
2. Files created
3. Files modified
4. Tests added or changed
5. Validation commands run and results
6. Unresolved issues
7. Suggested next step

The exact packet renderer may adjust whitespace, but heading order and semantic sections must remain stable.

11.1 Referenced context rendering rules

When selectedWorkUnit.agentContextRefs resolves to project records, render each supported record type with a stable prefix and compact details:

Requirement:

- Requirement [REQ-001]: <title> - <description>
Acceptance criteria: <criterion 1>; <criterion 2>

Decision:

- Decision [D001]: <decision>
Reason: <reason>
Impact: <impact>

Risk:

- Risk [RISK-001]: <title> - Severity: <severity>
Mitigation: <mitigation or "None recorded">

Assumption:

- Assumption [ASM-001]: <statement>
Reason: <reason or "None recorded">

Open Question:

- Open Question [Q001]: <question>
Impact: <impact>
Status: <status>
Answer: <answer or "Unanswered">

12. Canonical Mutation Rules

aiqt next mutates state.json and runlog.jsonl only. It must not mutate project.json.

- state.currentWorkUnitId is set to the selected ready work unit ID.
- state.currentMilestoneId is set to the selected work unit milestoneId.
- selected work unit status changes from ready to in_progress.
- selected milestone status changes to in_progress.
- state.projectStatus changes from planned to in_progress when a work unit starts.
- state.lastAgentPacket is set to packet metadata.
- state.nextRecommendedCommand becomes aiqt checkpoint.
- state.lastUpdatedAt is updated using the command timestamp.
- runlog.jsonl receives append-only events.
- No markdown packet file is created by default.
- AIQT_SCHEMA_VERSION remains 0.5.0 if lastAgentPacket metadata already fits the existing nullable metadata model.

13. State Transition Rules

In M4, a selected work unit must have status = ready. Therefore, under M3 status rules its parent milestone must already be ready. M4 does not transition planned milestones directly to in_progress.

Milestone status is a derived workflow summary, not a selection gate. When M4 selects one ready work unit inside a milestone, the selected work unit becomes in_progress and the milestone becomes in_progress. Other ready sibling work units remain ready. However, M4 must not generate another packet while state.currentWorkUnitId is non-null; a second aiqt next call before checkpoint must block with exit code 2 and recommend aiqt checkpoint.

Before	After	Allowed in M4
Project: planned	Project: in_progress	Yes, when a ready work unit is selected.

Milestone: ready	Milestone: in_progress	Yes, for the selected work unit milestone.
Milestone: planned	Milestone: in_progress	No. This transition is unreachable in M4 and reserved for future workflow rules.
Work unit: ready	Work unit: in_progress	Yes, for the selected work unit only.
Work unit: planned	Work unit: ready	No. M4 does not resolve dependencies.
Work unit: in_progress	Work unit: needs_review/done/blocked	No. M5 checkpoint handles this.
Any status	replanned/cancelled	No. Replanning remains out of scope.

M4 must not mark work as done. Completion requires a checkpoint.

14. Runlog Rules

A successful new packet generation appends two events in this order:

7. agent_packet.created
8. work_unit.status_changed

Failed or blocked aigt next attempts must not append runlog events.

```
{
  "id": "EVT-005",
  "type": "agent_packet.created",
  "timestamp": "ISO_DATE",
  "actor": "aiqt",
  "summary": "Agent packet created for work unit WU001.",
  "relatedIds": ["PROJECT-001", "M001", "WU001", "PKT-001"],
  "data": {
    "packetId": "PKT-001",
    "workUnitId": "WU001",
    "milestoneId": "M001",
    "format": "markdown",
    "contentHash": "sha256:<hash>",
    "nextRecommendedCommand": "aiqt checkpoint"
  }
}
```

```
{
  "id": "EVT-006",
  "type": "work_unit.status_changed",
  "timestamp": "ISO_DATE",
  "actor": "aiqt",
  "summary": "Work unit WU001 started.",
  "relatedIds": ["PROJECT-001", "M001", "WU001"],
  "data": {
    "workUnitId": "WU001",
    "fromStatus": "ready",
    "toStatus": "in_progress",
    "reason": "Selected by aigt next."
  }
}
```

Event IDs must continue from the highest existing EVT id. Packet IDs use the PKT- prefix and continue from the highest existing packet ID discoverable from state.lastAgentPacket and runlog events.

15. Command Behavior

15.1 aigt next

9. Locate .aiqt/.
10. Read project.json and state.json.
11. Validate schemas and versions.
12. Inspect runlog health using existing behavior.
13. If the work graph is empty, evaluate planningContextReady directly: recommend aigt update when false, recommend aigt plan when true, and return exit code 2 with no mutation.
14. Verify there is no currentWorkUnitId already in progress.
15. Select the first ready work unit by stored workUnits order.
16. Run the agent handoff gate.
17. Build bounded packet context.
18. Render the agent execution packet.
19. Compute packet metadata and content hash.
20. Update state.json according to M4 mutation rules.
21. Append runlog events.
22. Return the packet as human-readable output.

15.2 aigt next --json

JSON mode returns the existing CommandResult contract and exposes packet details under data. The packet string is included in data.packet.

```
{
  "status": "passed",
  "action": "next",
  "projectStatus": "in_progress",
  "currentMilestoneId": "M001",
  "currentWorkUnitId": "WU001",
  "summary": "Agent packet created for WU001.",
  "completedActions": ["Read project.json", "Read state.json", "Selected ready work unit", "Generated agent packet", "Updated workflow state", "Appended runlog events"],
  "changedFiles": [".aiqt/state.json", ".aiqt/runlog.jsonl"],
  "affectedItems": ["PROJECT-001", "M001", "WU001", "PKT-001"],
  "blockingIssues": [],
  "warnings": [],
  "requiresHumanInput": false,
  "nextRecommendedCommand": "aiqt checkpoint",
  "data": {
    "packetId": "PKT-001",
    "workUnitId": "WU001",
    "milestoneId": "M001",
    "packetFormat": "markdown",
    "contentHash": "sha256:<hash>",
    "packet": "# AGENT EXECUTION PACKET\n...",
    "statusChanges": [{"entityType": "workUnit", "id": "WU001", "from": "ready", "to": "in_progress"}]
  },
  "exitCode": 0
}
```

16. Data and Schema Contracts

16.1 AgentPacketMetadataSchema

```
export const AgentPacketMetadataSchema = z.object({
  id: z.string(),
  workUnitId: z.string(),
  milestoneId: z.string(),
  createdAt: z.string(),
  format: z.enum(["markdown"]),
  contentHash: z.string(),
  sourceCommand: z.literal("aiqt next"),
});
```

If the existing M1/M2/M3 implementation already has a narrower AgentPacketMetadataSchema, M4 may extend it only if the change is backward-compatible with existing null values and existing tests. Do not bump AIQT_SCHEMA_VERSION unless final implementation review identifies an incompatible canonical schema change.

16.2 PacketContext

```
export interface PacketContext {
  projectId: string;
  projectObjective: string;
  targetUsers: string[];
  milestone: Milestone;
  workUnit: WorkUnit;
  constraints: string[];
  technologyPreferences: string[];
  businessRules: string[];
  referencedRequirements: Requirement[];
  referencedDecisions: Decision[];
  referencedRisks: Risk[];
  referencedAssumptions: Assumption[];
  referencedOpenQuestions: OpenQuestion[];
  dependencies: Dependency[];
}
```

16.3 Packet ID prefix

Record type	ID prefix	Examples
Agent packet	PKT-	PKT-001, PKT-002
Runlog event	EVT-	EVT-005, EVT-006

17. Error Handling and Exit Codes

Case	Exit code	Status	Mutation	Next command
.aiqt/ missing	3	failed	No	aiqt init
project.json or state.json invalid	3	failed	No	null
unsupported future schema version	3	failed	No	null
work graph missing or empty and planningContextRea	2	blocked	No	aiqt update

dy is false				
work graph missing or empty and planningContextReady is true	2	blocked	No	aiqt plan
currentWorkUnitId already set to in_progress work unit	2	blocked	No	aiqt checkpoint
no ready work unit exists	2	blocked	No	aiqt review
selected work unit fails handoff gate	1	failed	No	aiqt review
selected work unit references invalid milestone/dependency IDs	3	failed	No	aiqt review
successful packet generation	0	passed	Yes	aiqt checkpoint

Exit code 1 is used when canonical state is valid but the selected work unit fails the quality/validation gate for packet generation. Exit code 2 is reserved for valid workflow-state blocks. Exit code 3 is reserved for invalid input or invalid canonical state.

18. Repository Implementation Scope

Recommended M4 implementation files added or modified:

```
src/
cli/
  commands/
    next.command.ts
  register-commands.ts
  options.ts
services/
  agent-packet-service.ts
templates/
  agent-packet.template.ts
workflow/
  next-work-unit-selector.ts
  agent-handoff-gate.ts
  status-transitions.ts
state/
  workflow-state-store.ts  # may be extended only if needed
  runlog-store.ts        # extend with M4 event builders
  ids.ts                 # extend for PKT IDs if needed
core/
  util/
    hash.ts              # optional, use Node crypto

tests/
unit/
  agent-packet-service.test.ts
  agent-handoff-gate.test.ts
  next-work-unit-selector.test.ts
  status-transitions.test.ts
integration/
```

next-packet.command.test.ts
next-checkpoint-flow.test.ts
fixtures/
planned-project/
planned-project-with-context-refs/
invalid-handoff-state/
no-ready-work-unit/

Follow the actual repository layout produced by M1-M3. Do not create duplicate helper locations if equivalent folders already exist.

19. Runtime and Package Impact

- Continue using Node.js, TypeScript, ESM, pnpm, commander, zod, and vitest.
- Do not add AI, GitHub, markdown-parser, graph-database, or source-code execution dependencies.
- Use Node crypto for packet content hashing if needed.
- Do not add clipboards in M4 v0.4. Clipboard support remains out of scope.
- package.json.version may remain unchanged unless implementation intentionally bumps package metadata.
- AIQT_SCHEMA_VERSION should remain 0.5.0 unless the final M4 implementation creates an incompatible canonical schema change.

20. Testing Scope

- All existing M1, M2, and M3 tests must continue to pass.
- Unit tests must cover ready work-unit selection, handoff gate pass/fail cases, packet rendering, context selection, status transitions, packet metadata, and runlog event builders.
- Integration tests must cover aiqt next success, aiqt next --json success, missing graph, no ready work unit, current work unit already in progress, invalid handoff state, and no markdown files created.
- Tests must confirm project.json is not mutated by aiqt next.
- Tests must confirm state.json and runlog.jsonl mutate only on successful new packet generation.
- Tests must confirm selected work unit moves ready -> in_progress.
- Tests must confirm currentWorkUnitId is set and nextRecommendedCommand becomes aiqt checkpoint.
- Tests must confirm generated packet includes required sections and excludes unrelated future work units.
- Tests must confirm JSON output follows deterministic CommandResult shape.
- Tests must confirm malformed runlog lines continue to warn according to M1/M2 behavior.

21. Implementation Tasks

Task	Scope	Acceptance
M4-T1 - CLI behavior	Upgrade next.command.ts from recommendation mode to packet mode.	aiqt next and aiqt next --json are registered and route through the packet engine.
M4-T2 - Work-unit selector	Select first ready work unit by stored order.	Unit tests cover multiple ready work units and no-ready cases.
M4-T3 - Handoff gate	Validate selected work unit and required packet inputs.	Gate failures return exit code 1 or 3 as specified, with no mutation.
M4-T4 - Packet context builder	Build bounded context from project/state and agentContextRefs.	Only relevant context is included; unresolved refs produce warnings only.
M4-T5 - Packet renderer	Render stable markdown-like packet.	Packet includes all required sections in deterministic order.
M4-T6 - State transitions	Move selected work unit to in_progress and update current IDs.	state.json changes are correct and project.json is unchanged.
M4-T7 - Packet metadata	Store lastAgentPacket metadata and packet hash.	PKT IDs increment and metadata validates.

M4-T8 - Runlog events	Append agent_packet.created and work_unit.status_changed.	Event IDs continue and event shapes are deterministic.
M4-T9 - Output rendering	Return packet in human mode and CommandResult in JSON mode.	Human mode is paste-ready; JSON mode includes data.packet.
M4-T10 - Tests and validation	Add M4 unit/integration coverage.	pnpm validate passes and M1-M3 behavior is preserved.

22. Definition of Done

- aiqt next can generate a bounded packet for one ready work unit.
- aiqt next --json returns a deterministic CommandResult with packet data.
- The selected work unit is marked in_progress.
- currentWorkUnitId is set to the selected work unit.
- currentMilestoneId is set to the selected work unit milestone.
- projectStatus becomes in_progress.
- selected milestone status becomes in_progress.
- state.nextRecommendedCommand becomes aiqt checkpoint.
- state.lastAgentPacket stores deterministic packet metadata.
- runlog.jsonl records packet creation and status change events.
- project.json is not mutated by aiqt next.
- No markdown files, milestone files, task files, AIQT.md, AGENTS.md, or CLAUDE.md are created.
- The packet includes objective, scope, out-of-scope, relevant context, constraints, dependencies, acceptance criteria, suggested files, validation commands, and required output contract.
- The packet does not include full project history or unrelated future milestones.
- Blocked and invalid cases return deterministic exit codes.
- pnpm typecheck, pnpm lint, pnpm test, pnpm build, and pnpm validate pass.
- M1, M2, and M3 tests still pass.
- The project is ready for Milestone 5: aiqt checkpoint.

23. Agent Execution Packet

AGENT EXECUTION PACKET

Role

You are implementing Milestone 4 of AIQT CLI.

AIQT is a local TypeScript CLI workflow state engine for projects built with AI coding agents.

Current Work Unit

ID: AIQT-M4

Title: Agent Handoff Packet Engine

Objective: Implement full aiqt next behavior that selects one ready work unit, generates a bounded agent execution packet, marks the work unit in_progress, and prepares the workflow for checkpoint capture.

Starting Point

M1 is complete and tagged: m1-canonical-state-foundation.

M2 is complete and tagged: m2-project-context-capture.

M3 is complete, merged to main at f922a47, and tagged: m3-work-graph-planning.

Start from main and create branch: feat/m4-agent-handoff-packet-engine.

Scope

Implement:

- aiqt next

- aiqt next --json

- first-ready-work-unit selection
- agent handoff gate
- bounded context selection
- agent packet renderer
- status transition ready -> in_progress
- currentMilestoneId/currentWorkUnitId update
- lastAgentPacket metadata
- agent_packet.created runlog event
- work_unit.status_changed runlog event
- deterministic human and JSON output
- tests for the M4 matrix

Out of Scope

Do not implement:

- aiqt checkpoint
- aiqt review
- aiqt export
- aiqt next --copy
- agent execution
- Claude/OpenAI API calls
- source-code modification
- markdown file generation
- AIQT.md, AGENTS.md, CLAUDE.md
- replanning
- checkpoint result capture

Validation Commands

pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate

Required Agent Output

Return:

1. Summary of implementation
2. Files created
3. Files modified
4. Tests added or changed
5. Validation commands run
6. Confirmation that M1, M2, and M3 tests still pass
7. Confirmation that no out-of-scope functionality was implemented
8. Any unresolved issues
9. Suggested next step

Appendix A - Minimum Test Matrix

Test Type	Scenario	Expected Result
Unit	Selector chooses first ready work unit by stored order	Selected WU ID is deterministic.
Unit	Selector ignores planned/in_progress/done work units	Only ready candidates are eligible.
Unit	Handoff gate passes for valid ready work unit	Pass.
Unit	Handoff gate rejects missing	Exit code 1 through command path.

	acceptance criteria	
Unit	Handoff gate rejects missing validationCommands	Exit code 1 through command path.
Unit	Packet renderer includes all required sections	Stable section order.
Unit	Packet renderer excludes unrelated work units	No unrelated future milestones in packet.
Unit	Context builder resolves agentContextRefs	Referenced records included.
Unit	Context builder handles unresolved refs	Medium context warning; packet generation proceeds.
Unit	PKT ID generation increments	Stable next ID.
Unit	Status transition ready -> in_progress	Only selected WU changes.
Integration	aiqt next on planned project succeeds	Packet printed; exit code 0.
Integration	aiqt next --json succeeds	CommandResult contains data.packet.
Integration	aiqt next without work graph and planningContextReady is false	Exit code 2, recommend aiqt update.
Integration	aiqt next without work graph and planningContextReady is true	Exit code 2, recommend aiqt plan.
Integration	aiqt next with currentWorkUnitId already set	Exit code 2, recommend aiqt checkpoint.
Integration	aiqt next with no ready work unit	Exit code 2, recommend aiqt review.
Integration	aiqt next invalid canonical state	Exit code 3.
Integration	aiqt next handoff gate failure	Exit code 1, no mutation.
Integration	successful next mutates state.json only	project.json unchanged.
Integration	successful next appends runlog events	agent_packet.created and work_unit.status_changed.
Integration	no markdown files created	No AIQT.md/AGENTS.md/CLAUDE.md/docs/.milestones/.tasks.
Integration	init -> update -> plan -> next flow	Project becomes in_progress and next command is aiqt checkpoint.

Appendix B - Example Agent Packet

AGENT EXECUTION PACKET

Role

You are implementing one bounded work unit for this project.

Project Objective

Build a local CLI workflow engine for AI agent implementation control.

Current Work Unit

ID: WU001

Milestone: M001 - Foundation

Title: Initialize application structure

Objective: Create the first runnable application shell.

Scope

- Create the application scaffold.
- Add package scripts required by the project.
- Add the initial source folder structure.

Out of Scope

- Do not implement authentication.
- Do not implement deployment.
- Do not create markdown documentation files by default.

Relevant Context

- Target users: Human project owner using AI coding agents.
- Technology preference: Node.js, TypeScript, pnpm.
- Business rule: Markdown is generated only on request.

Constraints

- Local file-based state only.

Dependencies

- None.

Acceptance Criteria

- Application starts successfully.
- TypeScript compiles successfully.
- Test command runs successfully.

Suggested Files / Areas

- package.json
- src/
- tests/

Validation Commands

```bash

pnpm typecheck

pnpm test

pnpm build

```

Required Agent Output

Return:

1. Summary of implementation
2. Files created
3. Files modified
4. Tests added or changed
5. Validation commands run and results
6. Unresolved issues
7. Suggested next step